LLM Agent Use Cases and Proposed Interfaces

1. Objective and scope

The agent will let authorized project users ask questions about application records, available predictions, dashboard summaries, model evaluations, and reports. It will retrieve evidence through validated agent tools backed by the dashboard data-access layer, then explain only what those results support.

The agent is an information and reporting interface, not a credit decision system. It must not approve or reject an application, fabricate application fields, predictions, metrics, or explanations, or make unsupported financial, causal, or historical claims.

This document defines proposed agent-facing tools. It does not implement tools, an agent, a graph, or a chat interface. The package under `src/agent/` is currently a set of empty placeholders; the six interfaces below are not implemented there.

2. Users and supported questions

Initial users are project analysts, data scientists, and engineering team members inspecting the platform's available data and model outputs. The agent supports factual questions answerable from retrieved project data, for example:

- "What application fields are available for application 1001?"
- "Is a prediction available for application 1001?"
- "How many applications have a valid target label?"
- "Which evaluation metrics are available for XGBoost and LSTM?"
- "What explanation did the prediction provider return?"
- "Summarize the available results for this reporting period."

The agent must identify synthetic data as mock/demo data and distinguish an absent result from a zero value or a negative finding.

3. Current implementation facts

The dashboard data-access contract is implemented in `src/dashboard/data_access.py`:

Existing function or type	Actual behavior
`get_provider_name() -> str`, `is_mock_mode() -> bool`	Provider selection reads `CREDIT_RISK_DATA_PROVIDER`; the default provider is `mock`. Only the mock provider is implemented. An unknown or empty provider name is not silently replaced with mock.
`get_data_provider() -> DashboardDataProvider`	Selects the mock implementation or raises `DataSourceUnavailableError`. The provider protocol exposes application, prediction, and evaluation retrieval.

Existing function or type	Actual behavior
`get_applications() -> pandas.DataFrame`	Returns a copy of the configured provider's records. An empty DataFrame is valid. An unavailable source raises `DataSourceUnavailableError`; a non-DataFrame response raises `MalformedProviderResponseError`.
`get_application_by_id(application_id, applications=None) -> ApplicationLookupResult`	Accepts a positive integer or a trimmed digit-only string; booleans, floats, zero/negative IDs, and other strings are invalid. Result status is `found`, `invalid_id`, `not_found`, `unavailable`, or `duplicate`. The `application` is a copied Series only for `found`. An empty source or missing `SK_ID_CURR` returns `unavailable`.
`get_overview_metrics(applications=None) -> dict[str, Any]`	Returns `total_applications`, `target_0`, `target_1`, `difficulty_rate`, `unlabeled`, and `risk_distribution`. Target counts/rate use only valid `TARGET` values 0 and 1. Missing or wholly unusable target data produces `None` for target counts/rate; a missing `RISK_CATEGORY` produces an empty distribution. If it loads the source itself, provider errors propagate.
`get*valid*target_values(applications) -> Series	None`
`get_credit_risk_prediction(application_id) -> CreditRiskPrediction`	Returns the existing Task 9 dataclass contract: `status`, `probability`, `model_name`, `model_version`, `prediction_timestamp`, `explanation`, and `message`. It validates the ID and provider response. Probability must be a non-boolean, finite number in `[0, 1]`. It returns explicit `invalid`, `unavailable`, `no_prediction`, `model_unavailable`, or `available` status.
`get*model*evaluation_results() -> DataFrame	None`
`normalize_model_evaluation_results(results)`	Validates the existing `model`, `metric`, `value` contract and returns `(values, statuses, issues)`. Supported models are XGBoost and LSTM; supported metrics are ROC-AUC, PR-AUC, Precision, Recall, and F1-score. Valid scores are finite values from 0 to 1. Missing, invalid, or duplicate rows remain unavailable and are reported in statuses/issues.
`DataSourceUnavailableError`, `MalformedProviderResponseError`	Explicit provider/source errors. Pages handle these errors and render an unavailable state rather than inventing results.

The mock provider currently creates synthetic application records with IDs 1001–1120, including `TARGET`, `RISK_CATEGORY`, and a synthetic `RISK_PROBABILITY` field. The latter is not a prediction response: the mock prediction method always reports unavailable. The mock provider also returns no evaluation results. Pages label mock data as demo data.

The Overview page calculates category, target-by-category, and numeric distributions from the loaded DataFrame for presentation; they are not all returned by `get_overview_metrics`. Model-comparison result normalization is already shared through `normalize_model_evaluation_results`. Application lookup and prediction display already use the data-access layer.

The `src/agent/` package currently has no implemented tool, report generator, prompt, state, or graph behavior. The proposed tools below are not existing Python functions. `src/dashboard/components/prediction_view.py` does not currently define a prediction interface.

4. Use cases

Use case	Example questions	Required input	Proposed tool	Expected response	Possible failure or unavailable state
Application details	"Show the available details for application 1001." "Which fields are missing?"	One positive application ID	`get_application_details(application_id)`	Found/not-found status and only the fields returned for that record. Identify null fields separately from fields not present in the record; do not infer missing values.	Invalid ID, not found, duplicate ID, unavailable source/ID column, malformed provider response, or no fields to display.
Credit-risk prediction	"Is a prediction available for application 1001?" "What probability and model version were returned?"	One positive application ID	`get_prediction(application_id)`	Provider-supplied prediction status and, when available, probability, model name, model version, and timestamp. Describe probability as the likelihood of `TARGET=1` only when that target interpretation is confirmed by the inference contract.	Invalid ID, no prediction, prediction provider unavailable, model unavailable, malformed response, or missing optional metadata.
Dashboard metrics	"How many applications have valid targets?" "What are the target counts and available risk categories?"	No required input; optional scope only if the source supports it	`get_dashboard_metrics()`	Counts and rates returned by the shared metrics function, with unavailable values represented as unavailable rather than zero. Risk-category counts are available only when that field exists.	Provider unavailable, empty dataset, missing/invalid `TARGET`, absent risk field, malformed response, or mock-data source.
Model comparison	"Compare the available XGBoost and LSTM metrics." "Which metrics are missing?"	No required input; optional period only if evaluation outputs support filtering	`get_model_comparison()`	Available supported metrics and per-model availability/partial-result status. Never fill missing metrics with estimates.	No evaluation results, one model absent, partial/invalid/duplicate metric rows, unavailable provider, malformed result type, or LSTM evaluation not available.
Prediction explanation	"What explanation was returned for application 1001?"	One positive application ID	`explain_prediction(application_id)`	Display only explanation payloads returned by the prediction provider, with the related prediction/model metadata when available.	Invalid ID, prediction unavailable, explanation missing, unsupported explanation format, model unavailable, or provider unavailable.
Report generation	"Summarize the available model results for the selected period." "Create a current portfolio summary."	A requested period/scope string; it must be supported by the underlying source to filter data	`generate_report(period)`	Structured summary of retrieved application metrics and/or model results, with source, mock/real label, coverage, and unavailable sections identified.	Period not supported by source data, missing timestamps, unavailable inputs, partial results, or no evidence for requested trends.

`TARGET=1` means repayment difficulty; `TARGET=0` means no defined repayment difficulty. Neither label is an approval decision. A target distribution must show its labeled denominator and not treat missing or invalid target values as either class.

5. Proposed agent-facing tool interfaces

These are proposed Python-level wrappers. They should return JSON-compatible objects for agent consumption rather than expose pandas objects or Streamlit elements. The structures below define the intended wrapper contract; they are not existing return types. Existing APIs and their return values are listed in Section 3. Agent wrappers must reuse those APIs rather than load data or reimplement calculations.

All tools should return only retrieved values plus explicit status/provenance metadata. They should not turn exceptions into successful-looking empty or zero results. At minimum, a wrapper should distinguish a successful result, an unavailable result, invalid input, and malformed source data, either through a documented result status or a raised, handled tool error.

`get_application_details(application_id)`

- **Purpose:** Retrieve one application and summarize its available fields.
- **Input:** `application_id: int | str`. Apply the data-access validation:

positive integer or trimmed digit-only string; reject booleans, floats, and non-positive values.

- **Source/mapping:** `get_application_by_id(application_id)`, which calls

`get_applications()` when no DataFrame is supplied. Do not query files or storage from the agent.

- **Proposed output:** `{status, application_id, fields, null_fields, message}`.

`fields` contains only source-provided, non-null values. `null_fields` lists source-provided fields whose value is null. Do not claim that a field absent from the row is null unless a source schema identifies it as expected.

- **Validation/errors:** Preserve lookup statuses `invalid_id`, `not_found`,

`duplicate`, and `unavailable`; catch and report `DataSourceUnavailableError` / `MalformedProviderResponseError` as tool errors. Never select one duplicate record arbitrarily.

- **Status:** Implementable now as a thin adapter using the existing lookup

function. The JSON-safe conversion and missing/null presentation are proposed wrapper behavior, not currently implemented.

`get_prediction(application_id)`

- **Purpose:** Retrieve one application's provider-supplied prediction.
- **Input:** `application_id: int | str`, validated as above.
- **Source/mapping:** `get_credit_risk_prediction(application_id)`.
- **Proposed output:** The `CreditRiskPrediction` fields:

`{status, probability, model_name, model_version, prediction_timestamp, explanation, message}`. Preserve `None` for absent optional values. The probability is a number in `[0, 1]`; render as a percentage only for display.

- **Validation/errors:** Preserve `invalid`, `unavailable`, `no_prediction`,

`model_unavailable`, and `available`. The normalizer rejects missing, boolean, non-numeric, non-finite, and out-of-range probabilities as invalid. Do not substitute the mock record's `RISK_PROBABILITY`.

- **Status:** The retrieval boundary and response validation exist now, but

the configured mock provider returns `unavailable`. Real predictions, target meaning, model identity/version, and timestamps depend on Person 1's inference provider and its agreed response contract.

`get_dashboard_metrics()`

- **Purpose:** Return application counts, target distribution, and supported

risk statistics.

- **Input:** None in the initial interface. A date/filter argument is not

supported until the provider exposes a matching filterable dataset.

- **Source/mapping:** `get_applications()` followed by

`get_overview_metrics(applications)` so the page snapshot is reused. Do not reimplement target counts or read data independently.

- **Proposed output:** The current metrics mapping:

`{total_applications, target_0, target_1, difficulty_rate, unlabeled, risk_distribution}`. `difficulty_rate` is the share of valid labeled rows with `TARGET=1`; `risk_distribution` contains category counts. `None` means target statistics are unavailable; an empty risk distribution means no supported risk category was present. A wrapper may add a target share breakdown only when it is calculated from these retrieved counts and clearly identifies its denominator.

- **Validation/errors:** Empty data yields zero total and unavailable target

statistics, not fabricated observations. Missing columns are handled as described in Section 3. Report data-source errors explicitly.

- **Status:** Implementable now for whole-source metrics. Time periods,

streaming freshness, and metrics beyond the current returned mapping require additional provider contracts.

`get_model_comparison()`

- **Purpose:** Return actual available evaluation results for XGBoost and LSTM.
- **Input:** None initially. Period filtering is not supported by the current

evaluation retrieval function.

- **Source/mapping:** `get_model_evaluation_results()` followed by

`normalize_model_evaluation_results(results)`.

- **Proposed output:** `{models, metrics, statuses, issues}`. `models` contains

XGBoost and LSTM; `metrics` contains ROC-AUC, PR-AUC, Precision, Recall, and F1-score values or null; `statuses` and `issues` preserve the shared normalizer's actual results. Scores are finite values from 0 to 1.

- **Validation/errors:** `None` or an empty result means no evaluation

results. Missing required columns, unknown identifiers, duplicate rows, and invalid scores must remain visible as invalid/partial status and issues. Distinguish unavailable provider errors from valid absence of evaluation rows.

- **Status:** Implementable now as an adapter to existing retrieval and

normalization functions. The current mock provider has no evaluation data; actual scores and LSTM evaluation eligibility depend on Person 1.

`explain_prediction(application_id)`

- **Purpose:** Retrieve and communicate a supported explanation for an

application's prediction.

- **Input:** `application_id: int | str`, validated as above.
- **Source/mapping:** Initially call `get_credit_risk_prediction()` and read

its provider-supplied `explanation` field. There is no separate explanation retrieval API or standardized explanation schema today.

- **Proposed output:** `{status, application*id, model*name, model_version,

explanation, message}`. Return` available` only when the prediction is available and a supported non-empty explanation payload is supplied.

- **Validation/errors:** If a prediction or explanation is unavailable,

return that state explicitly. Render only source-provided explanation content. Do not infer feature importance, causal effects, directionality, or SHAP values from the prediction probability or application fields.

- **Status:** The prediction response has an untyped optional `explanation:

Any` field, but an explanation tool is not implemented. A stable, documented explanation payload and corresponding provider output are dependencies before this tool can provide validated explanations.

`generate_report(period)`

- **Purpose:** Produce a structured narrative from retrieved, supported

application metrics and/or model evaluation results.

- **Input:** `period: str`, identifying the requested scope. The tool must

validate that the source can actually select that period; a label supplied by the user is not proof that data was filtered.

- **Source/mapping:** Compose only outputs from `get_dashboard_metrics()` and

`get_model_comparison()` (and other validated tools added later). No report generator currently exists.

- **Proposed output:** `{requested*period, data*coverage, source_mode,

application*metrics, model*comparison, limitations, summary}`. Each section may be explicitly unavailable; summary` must be grounded in the returned sections.

- **Validation/errors:** If the source has no relevant timestamps or period

filter, mark the period unsupported and offer an all-available-data summary only if the user accepts that scope. Do not claim trends, changes over time, or period comparisons without date-scoped records or evaluation artifacts. Preserve partial results and their missing sections; do not fill gaps.

- **Status:** Not implementable as a period report with current APIs. Whole

dataset summaries can be composed after the report wrapper is implemented; period filtering and trend claims depend on timestamped, filterable source data and evaluation coverage from Person 1.

6. Response, safety, and unavailable-data rules

The agent must apply these rules consistently, regardless of phrasing:

Condition	Required behavior
Invalid application ID	State that the ID is invalid and provide the accepted positive-integer format. Do not call it not found.
Application not found	State that no matching record was returned. Do not infer why it is absent.
Duplicate application ID	State that the ID is ambiguous and do not select a record.
Missing application fields	Show available fields only. Distinguish source nulls from fields not present; never guess or fill values.
Prediction unavailable	Say no prediction provider/result is available. Do not use `RISK_PROBABILITY`, `TARGET`, or an LLM estimate as a substitute.
Model unavailable	Identify the model status as unavailable; do not imply it was evaluated or that it performed poorly.
Missing evaluation metrics	Mark the specific metric unavailable. For partial results, state which metrics/models are present and absent. Never interpolate or supply example scores as real results.
Explanation unavailable	State that no supported explanation was returned. Do not generate feature-importance claims from the model score or record fields.
Data provider unavailable	Report the configured source/provider failure. Do not silently switch to mock or present empty data as a successful query.
Malformed provider response	Report a data/tool error and do not interpret malformed values as valid.
Partial results	Answer only the supported portion and list relevant limitations or unavailable fields.
Mock-data mode	Prominently label results synthetic/demo. Never describe mock records, risk categories, probabilities, predictions, or metrics as real applicants or real model output.

Use "repayment difficulty" for `TARGET=1` and "no defined repayment difficulty" for `TARGET=0`. Explain that target labels are historical outcomes, not lending decisions. The agent must not make automatic loan approval or rejection recommendations. It must not claim financial causality or unsupported conclusions. Factual answers may use only retrieved project information and clearly identified calculations derived from it.

7. Framework selection

Recommendation: LangGraph for the initial agent workflow. LangChain offers useful model, prompt, and tool abstractions; LangGraph adds explicit state and control over the sequence and branches of a tool-using workflow. The initial requirements prioritize validated inputs, allowlisted tool execution, visible unavailable/error paths, and grounded responses over open-ended autonomous planning.

A single agent and a small explicit graph are sufficient:

1. Receive a user question and identify the requested operation.
2. Validate required arguments and reject unsupported scopes.
3. Select from the six allowlisted tools and execute only the necessary calls.
4. Validate tool status/payloads, retaining provenance and unavailable fields.

5. Produce a concise response grounded in those outputs and enforce the

response rules in Section 6.

Tool calls should use typed schemas and the proposed wrapper contracts above. Do not expose filesystem, shell, arbitrary SQL, or unrestricted data access to the language model. Tool exceptions and provider errors should map to explicit error states, not success-shaped defaults. If a necessary tool is unavailable, the response should say so instead of attempting an unsupported alternate calculation.

Multiple agents are unnecessary for the first version: these use cases share one data-access boundary, have a small fixed tool set, and require consistent validation and response policy. Multiple agents would add routing and coordination complexity without a demonstrated independent responsibility. LangChain may be used for compatible model/tool components if useful, but a second orchestration framework is not a requirement. No framework dependency is installed or selected in code by this documentation task.

8. Intended architecture

```
Streamlit chat interface
        |
        v
LangGraph single-agent workflow
        |
        v
Validated, allowlisted agent tools
        |
        v
dashboard.data_access
        |
        v
Project data and ML interfaces
        |
        v
Retrieved results + provenance/status
        |
        v
Grounded agent response
```

The Streamlit chat interface and graph are future work. The agent must not independently read datasets, model artifacts, or storage, and must not duplicate the dashboard's retrieval or metric logic. Any new source should be connected through a provider/data-access contract, then exposed through a narrow validated agent tool.

9. Person 1 integration requirements

Before real-data use, Person 1 should agree and provide:

- **Application data:** Stable provider availability, identifier uniqueness,

application field/schema documentation, null semantics, and provenance for each source. A stable real provider must implement the existing `DashboardDataProvider` contract or an agreed compatible adapter.

- **Predictions:** A callable inference boundary by application ID; an agreed

response schema with probability meaning (`P(TARGET=1)`), model name/version, timestamp, explicit no-prediction/model-unavailable states, and malformed response behavior. Probability must be finite and in `[0, 1]`.

- **Model evaluation:** Evaluation artifact location/adapter, stable model and

metric identifiers, finite scores, evaluation dataset/split and timestamp, and explicit absent/partial-result semantics. Provide LSTM metrics only when valid repayment sequences and an evaluation were actually constructed.

- **Explanations:** A supported output source and documented schema (for

example, provider-generated SHAP values with feature names, values, units, and any supported interpretation limits). The agent must not infer the explanation format or claim SHAP support until supplied.

- **Periods and reports:** Timestamp fields, timezone/coverage semantics, and

provider-side filtering or aggregation if reports are to describe a period or historical change. Current overview/evaluation functions accept no period.

- **Source provenance:** A way to identify mock/demo versus real source and,

for real outputs, the relevant model/data version where available.

Person 2 owns the eventual agent wrappers, LangGraph workflow, prompt and response rules, error mapping, reporting composition, and their tests. The agent should integrate only through agreed interfaces, not ML internals.

10. Explicit out of scope

- Implementing the agent, graph, nodes, tool-calling code, prompts, or chatbot

UI.

- Training, modifying, or deploying XGBoost or LSTM.
- Generating predictions, model metrics, feature explanations, or application

data with an LLM.

- Automatic credit approval/rejection or lending recommendations.
- AWS, database, Kafka, or external-service integration work.
- Period trends, causal analysis, or unsupported financial advice.
- Multi-agent orchestration, unrestricted query tools, or unnecessary

autonomous actions.

11. Unresolved interfaces

The following decisions must be settled before the dependent tools are treated as production-ready: the real provider adapter and provenance schema; whether prediction probability is formally `P(TARGET=1)`; the model/version and timestamp guarantees; a stable explanation payload; LSTM sequence/evaluation availability; and whether application/evaluation data supports period filtering. Until then, dependent results must remain explicitly unavailable.